Lab6 Report

Name: 时分

Student ID: PB23151799

Purpose

The purpose is to realize implement temporal multithreading for two LC-3 programs A and B.

The user program A (located at `x4090`) and B (located at `x8090`) will be created and loaded at test time. When key `Q` is pressed, halt the machine. Whenever a program task switching via `TRAP`, serve it. The trap vector for switching is defined as `x77`.

The user programs won't require any input, won't call halt and won't use `R6` or `R7`. The code starts at x800 and runs in supervisor mode.

A boilerplate is given. Our task is to finish some parts of it: setup program, keyboard interrupt handler and trap handler.

Program Design

The first part is Entry, starting at `x200`. LC-3 sets PC to here at the beginning. This part is given in the boilerplate, by using simply `JMP` instruction.

The next part is Setup program, starting at `x800`. This part includes:

1. Enable keyboard interrupt. `KBSR[14]` is the interrupt enable bit (IE). We can set it to 1 by `KBSR` OR `x4000`. The assembly code is:

```
;Enable keyboard interrupt
LDI R0, KBSR
LD R1, KBSRMask
NOT R0, R0
NOT R1, R1
AND R0, R0, R1
NOT R0, R0
;R0 = KBSR OR x4000
STI R0, KBSR
;KBSR[14] = 1

KBSR .FILL xFE00
KBSRMask .FILL x4000
```

2. Sets handler for keyboard interrupt. Keyboard interrupt uses vector `x80`, so we need to put the starting address of the ISR (`x1000`) at `x0180`. The assembly code is:

```
;Setup ISR for keyboard
LD R0, KBISRAddr
STI R0, KBVecAddr

KBVecAddr .FILL x0180
KBISRAddr .FILL x1000
```

3. Sets handler for `TRAP x77`. We need to put the starting address of the trap handler (`x1800`) at `x0077`. The assembly code is:

```
;Setup handler for trap
LD R0, TrapISRAddr
STI R0, TrapVecAddr

TrapVecAddr .FILL x0077
TrapISRAddr .FILL x1800
```

4. Call user program A at `x4090`. This is realized by instruction `RTI` , which leaves the supervisor mode by changing PSR. We need to prepare two values for PC and PSR, push them into the stack (the pointer is `R6`) and call `RTI`. The value for PC is the starting address of program A (`x4090`). To change to user mode, `PSR[15]` should be 1. `PSR[10:8]` is the priority level, which should be 0. `PSR[2:0]` are condition codes, whose default values are `010`. So the value for PSR is `x8002`. The assembly code is:

```
;Calls user program A
LD R0, PSR_A
ADD R6, R6, x-1
STR R0, R6, x0
;Push PSR of program A into the stack
LD R0, PC_A
ADD R6, R6, x-1
STR R0, R6, x0
;Push PC of program A into the stack
RTI
```

Keyboard interrupt service routine starts at `x1000`. This part checks the pressed key. If the key is `Q`, halt the machine. The keycode striked is stored in `KBDR`, locating at `xFE02`. When `KBDR` is read, the ready bit (`KBSR[15]`) will be set to 0 automatically, and the IE bit won't be changed. The assembly code is:

```
; Keyboard ISR
.ORIG x1000

;Store R0, R1
ST R0, SAVE_R0
ST R1, SAVE_R1

;Get the keycode
LDI R0, KBDR

;If Q is pressed, halt the machine
LD R1, NEG_Q
ADD R1, R0, R1
BRnp SKIP
```

```
HALT
SKIP

;Restore R0, R1
LD R0, SAVE_R0
LD R1, SAVE_R1
RTI

SAVE_R0 .BLKW 1
SAVE_R1 .BLKW 1
NEG_Q .FILL x-51
KBDR .FILL xFE02

.END
```

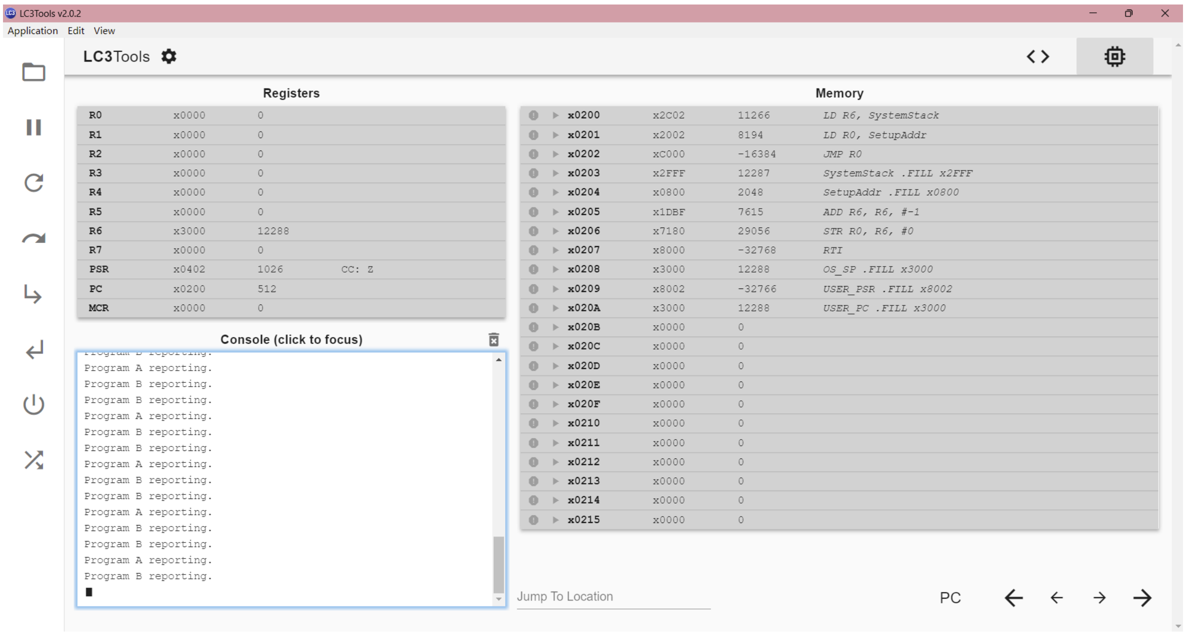
`TRAP x77` handler starts at `x1000`. This part calls program switch, which is given in the boilerplate. `R0` and `R1` are already restored in the keyboard ISR. The assembly code is:

```
; Trap x77 handler
.ORIG x1800

;Call switch program
LD R7, SwitchProgramAddr
JMP R7

SwitchProgramAddr .FILL x2000
.END
```

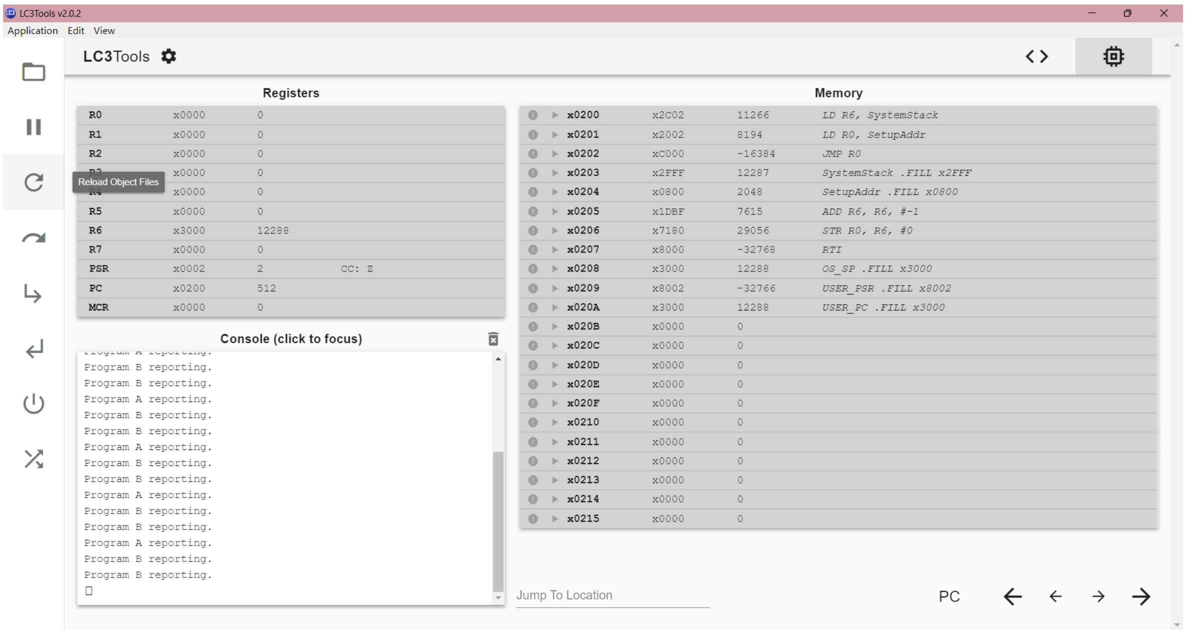
Program switch (starting at `x2000`) and the user programs are given in the boilerplate.

Testing Evidence (Screenshots)

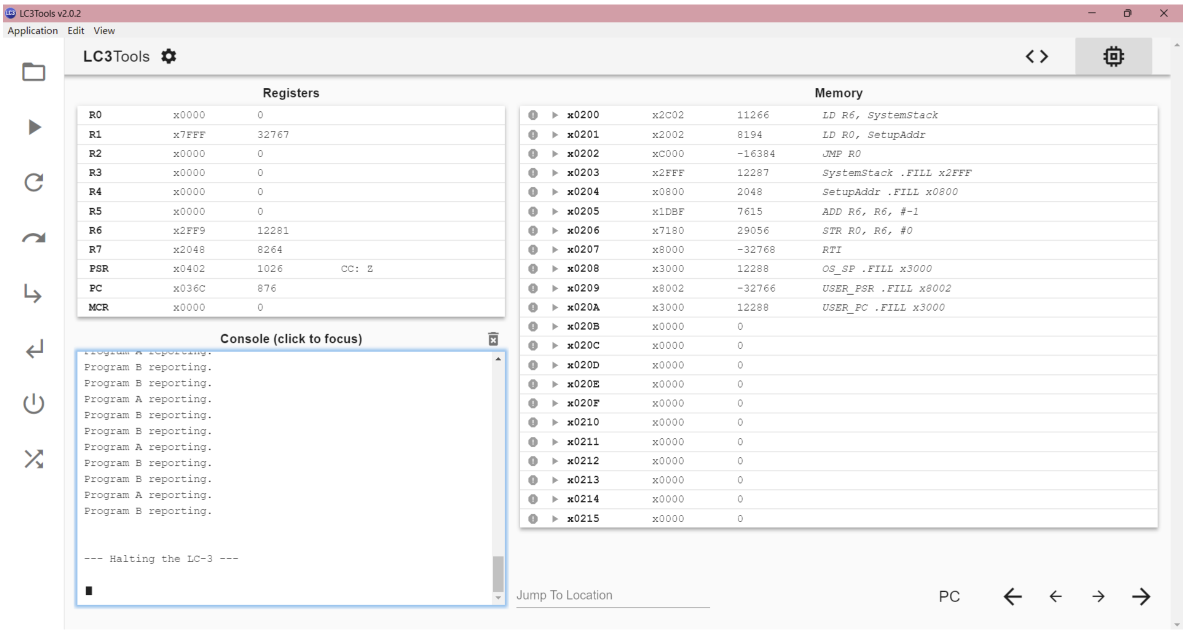
The program runs:



When 'B' is striked first:



When 'Q' is striked next:



Questions

1. Talk about how you understand that we must not change any register in the ISR.

   A: The ISR is called when user program is running. If any register is changed in the ISR, when LC-3 returns to user program, the program won't work properly. What's more, if the stack pointer `R6` is changed, poping and pushing process of the system stack and user stack won't work properly, either.

2. Using the program above, you may notice that if you hit some key (other than Q) quickly, LC-3 will crash. Suggest some possible cause of this.

   A: If the key is hit during the keyboard ISR, another keyboard ISR will be called, and the system stack will grow. When the key is hit too quick, the system stack may overflow, causing LC-3 to crash.

3. Implementing temporal multithreading is cumbersome in terms of scheduling, while for simultaneous multithreading things will be tricky when managing data synchronization. You may have also noticed that the program switch is very long (~100 lines). Paying such a highprice, why computers of nowadays still implement multithreading?

A:

1. TMT allows CPU to do more tasks at the same time, especially when there are more tasks need to be done at the same time than the cores.
2. When a program is waiting when performing operations like I/O, the CPU may be idle. TMT allows CPU to do other tasks during this time, which increases the efficiency.